

Методы `split` и `join`. Списочные выражения.

В данном занятии рассматриваются два очень популярных метода строк — `split` и `join`. Во второй части рассматриваются списочные выражения (python list comprehensions `[... for ... in ...]`), которые в сочетании с методом `split` позволяют, например, считывать из стандартного потока ввода несколько значений одной строкой.

Методы `split` и `join`

Изучая множества и списки, мы уже неоднократно встречались с методами — функциями, «приклеенными» к объекту (списку или множеству) и изменяющими его содержимое.

Методы есть не только у списков и множеств, но и у строк. Сегодня мы изучим два очень полезных метода строк — `split` и `join`. Они противоположны по смыслу: `split` разбивает строку по произвольному разделителю на список «слов», а `join` собирает из списка слов единую строку через заданный разделитель.

Чтобы вызвать эти методы, необходимо использовать уже знакомый нам синтаксис. После имени переменной, содержащей объект-строку, или просто после строки через точку пишется имя метода, затем в круглых скобках указываются аргументы. `split` и `join`, в отличие, например, от метода списков `append` или метода множеств `add`, не изменяют объект, которому принадлежат, а создают **новый** (список или строку) и возвращают его, как это делают обычные функции типа `len`.

Метод `split`

Метод `split` можно вызвать вообще без аргументов или с одним аргументом-строкой. В первом случае строка разбивается на части, разделенные любыми символами пустого пространства (набором пробелов, символом табуляции и т. д.). Во втором случае разделителем слов считается строка-аргумент. Из получившихся слов формируется список.

В этом примере все сравнения истинны, т. е. все вызовы функции `print` выведут `True`.

```
s = 'раз два три'
print(s.split() == ['раз', 'два', 'три'])
print('one two three '.split() == ['one', 'two', 'three'])
print('192.168.1.1'.split('.') == ['192', '168', '1', '1'])
print(s.split('a') == ['р', 'з дв', ' три'])
print('A##B##C'.split('##') == ['A', 'B', 'C'])
True
True
True
True
True
```

Метод `join`

join же всегда принимает один аргумент — список слов, которые нужно **склеить**. Разделителем (точнее, «соединителем») служит та самая строка, чей метод *join* вызывается. Это может быть и пустая строка, и пробел, и символ новой строки, и что угодно еще. В этих примерах все сравнения тоже истинны и каждый *print* выведет True.

```
s = ['Тот', 'Кого', 'Нельзя', 'Называть']
print(''.join(s) == 'ТотКогоНельзяНазывать')
print(' '.join(s) == 'Тот Кого Нельзя Называть')
print('-'.join(s) == 'Тот-Кого-Нельзя-Называть')
print('! '.join(s) == 'Тот! Кого! Нельзя! Называть')
True
True
True
True
```

Итак, *split* служит для преобразования строки в список, а *join* — для преобразования списка в строку:

Обратите внимание: *split* и *join* — методы **строк**. Попытка вызвать такой метод у объекта, не являющегося строкой, вызовет ошибку! Например, если попытаться написать заведомо бессмысленное, с точки зрения интерпретатора Python, выражение:

```
[1, 2, 3].join([4, 5, 6])
AttributeError                                Traceback (most recent call last)
<ipython-input-3-404b5f4d9169> in <module>()
----> 1 [1, 2, 3].join([4, 5, 6])
```

```
AttributeError: 'list' object has no attribute 'join'
```

Как видно, указан тип объекта ([1, 2, 3] — список, *list*) и имя отсутствующего у него метода — *join*, который мы пытаемся вызвать.

Списочные выражения

Для генерации списков из неповторяющихся элементов в Python имеется удобная синтаксическая конструкция — **списочное выражение** (list comprehension). Она позволяет создавать элементы списка в цикле *for*, не записывая цикл целиком.

Например, если нам необходимо создать список квадратов целых чисел от 0 до 9 включительно, мы можем записать следующий код:

```
squares = []
for i in range(10):
    squares.append(i ** 2)
print(squares)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

То же самое, но гораздо короче можно сделать с помощью списочного выражения:

```
squares = [i ** 2 for i in range(10)]
print(squares)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

А если нам необходимы квадраты не всех чисел, а только четных? Тогда можно добавить условие:

```

even_squares = []
for i in range(10):
    if i % 2 == 0:
        even_squares.append(i ** 2)
print(even_squares)
[0, 4, 16, 36, 64]

```

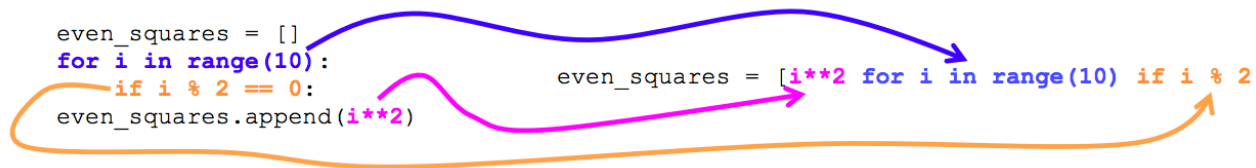
То же самое, но короче, с помощью списочного выражения:

```

even_squares = [i ** 2 for i in range(10) if i % 2 == 0]
print(even_squares)
[0, 4, 16, 36, 64]

```

На рисунке показано соответствие между кодом заполнения списка с помощью цикла *for* и списочным выражением:



В списочном выражении можно пройти по двум или более циклам:

```

print([i * j for i in range(3) for j in range(3)])
[0, 0, 0, 0, 1, 2, 0, 2, 4]

```

На самом деле квадратные скобки не являются неотъемлемой частью списочного выражения. Если их не поставить, выражение будет вычисляться по мере надобности — когда очередной элемент становится нужен. Заклучая списочное выражение в квадратные скобки, мы тем самым даем инструкцию сразу создать все элементы и составить из них список. Пока что мы в основном будем пользоваться именно такими списочными выражениями — заключенными в квадратные скобки и превращенными таким образом в список.

Использование списочных выражений

Списочные выражения часто используются для инициализации списков. Дело в том, что в Python не принято создавать пустые списки, чтобы потом заполнять их значениями, если можно этого избежать.

Если все-таки необходимо создать пустой список (скажем, длиной 10) и заполнить его нулями (не может же он быть совсем пустой), это легко сделать, используя умножение списка на число: `[0] * 10`.

Использование списочных выражений в аргументах методов *split* и *join*. Считывание значений, введенных одной строкой

Списочные выражения часто используют в аргументах методов *split* и *join*. Например, комбинация метода *split* и списочного выражения позволяют нам удобно считать числа, записанные в одну строку:

```

a = [int(x) for x in '976 929 289 809 7677'.split()]
evil, good = [int(x) for x in '666 777'.split()]
print(evil, good, sep='\n')
666
777

```

Здесь строка (обычно она не задается прямо в выражении, а получается из `input()`) разделяется на отдельные слова с помощью *split*. Затем списочное выражение пропускает каждый элемент получившегося списка через функцию *int*, превращая строку '976' в число 976. Можно собрать все получившиеся значения в один список или разложить их по отдельным переменным с помощью множественного присваивания, как во второй строчке примера.

Рассмотрим и пример использования метода *join* вместе со списочным выражением. Выведем на одной строке список квадратов натуральных чисел от 1 до 9: $1^2=1$, $2^2=4$, $3^2=9$... Для этого сначала с помощью списочного выражения сформируем список строк вида ['1^2=1', '2^2=4', '3^2=9', ...], а затем «склеим» его в одну строку методом *join*:

```
print(','.join(str(i) + '^2=' + str(i ** 2) for i in range(1, 10)))
```

1^2=1, 2^2=4, 3^2=9, 4^2=16, 5^2=25, 6^2=36, 7^2=49, 8^2=64, 9^2=81

Заметьте, что в аргументе функции *join* стоит списочное выражение, не заключенное в квадратные скобки (это можно сделать, но необязательно).

Будьте внимательны! Обычно возведение в степень обозначают «крышечкой» перед степенью, но в Python эта «крышечка» обозначает совсем другое, а возведение в степень выполняется оператором `**`. В примере «крышечка» используется только как обозначение операции возведения в степень.

Пример 1. Горизонтальная диаграмма

Ограничение времени	1 секунда
Ограничение памяти	64Mb
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Напишите программу для построения горизонтальных столбчатых диаграмм с помощью символа звёздочки. Подсказка: выводя строку, не используйте для этого цикл.

Формат ввода

Вводится несколько натуральных чисел, разделённых пробелами.

Формат вывода

Для каждого введенного числа выводится строка, состоящая из звёздочек *, длина которой равна этому числу.

Пример

Ввод	Вывод
3 7 1 10 8	*** ***** * ***** *****

```
1 a = input().split()
2 for i in a:
3     print('*' * int(i))
4 |
```

Пример 2. /etc/passwd

Ограничение времени	1 секунда
Ограничение памяти	64Mb

Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Исторически сложилось, что в Unix-подобных операционных системах информация о пользователях хранится в файле под названием `/etc/passwd`. Формат этого файла такой: информация о каждом пользователе записана на отдельной строке и состоит из семи полей, разделённых двоеточиями:

- логин пользователя;
- пароль пользователя (на самом деле не совсем так, но в этой задаче будем считать, что там действительно хранится пароль);
- номер пользователя в системе;
- номер группы, к которой принадлежит пользователь;
- дополнительная информация о пользователе;
- домашний каталог пользователя;
- интерпретатор командной строки, который запускается для пользователя.

Нас здесь будут интересовать первое, второе и пятое поля.

Предположим, вы — системный администратор, и хотите порекомендовать тем из ваших сотрудников, кто использует слишком распространённый пароль, сменить его. У вас имеется файл `/etc/passwd` с сервера с информацией обо всех пользователях. У вас принято, что дополнительная информация о пользователе системы начинается с его имени, а затем после запятой может следовать любая другая информация (например, должность). Кроме того, у вас имеется список самых часто используемых паролей. Напишите тем сотрудникам, кто использует один из этих паролей, чтобы они сменили его.

Формат ввода

Сначала вводится содержимое файла `/etc/passwd` в описанном в условии формате (не менее одной строки).

Затем следует пустая строка.

После этого вводится список самых часто используемых паролей (не менее одного), разделённых точкой с запятой.

Формат вывода

Для каждого сотрудника, пароль которого — один из самых часто используемых, выведите две строки. На первой строке после слова «То: » укажите логин пользователя, чтобы потом другая программа отправила ему ваше сообщение по электронной почте. На второй строке выведите: «<Имя сотрудника>, ваш пароль слишком простой, смените его.»

Пример

Ввод

```
ivanov:qwerty:100:1:Сергей Иванов, менеджер:/home/ivanov:/bin/sh
ilyina:gfghjkm:101:1:Мария Ильина, старший программист, HL3:/home/ilyina:/bin/sh
kuznetsova:jxty[m[nhsqgfghjkm:102:1:Дарья Кузнецова, младший программист:/home/kuznetsova:/bin/s
polivanov:qwerty:103:1:Андрей Поливанов, младший программист, TF3:/home/polivanov:/bin/sh

123456;qwerty;password
```

```

1 a = []
2 b = []
3 c = []
4 d = []
5 while (True):
6     d.clear()
7     e = input()
8     if e == '':
9         break
10    else:
11        d = e.split(':')
12        f = d[4].split(',')
13        b.append(d[0])
14        a.append(f[0])
15        c.append(d[1])
16    g = input().split(';')
17    for i in range(len(c)):
18        if c[i] in g:
19            print('To:', b[i])
20            print(a[i], ', ваш пароль слишком простой, смените его.', sep='')
21

```

Пример 3. Только без лука!

Ограничение времени	1 секунда
Ограничение памяти	64Mb
Ввод	стандартный ввод или input.txt
Вывод	стандартный вывод или output.txt

Предположим, вы переписываете у друга рецепты в блокнотик, но вам не нравится лук. Переписывайте без него.

Формат ввода

На первой строке вводится натуральное число N — количество пунктов рецепта. Далее следуют N строк — пункты рецепта.

Формат вывода

Одна строка — пункты рецепта, разделённые запятой и пробелом, без пунктов с упоминанием лука (то есть таких, в которых нет подстроки "лук" в нижнем регистре).

Пример

Ввод	Вывод
5 лук 1 головка картофелин штук 6 картошку почистить лук порезать кольцами зажарить всё	картофелин штук 6, картошку почистить, зажарить всё

```

1 q = int(input())
2 a = []
3 for i in range(q):
4     s = input()
5     if 'лук' not in s:
6         a.append(s)
7 print(', '.join(a))

```

Пример 4. Вертикальная диаграмма

Ограничение времени	1 секунда
Ограничение памяти	64Mb
Ввод	стандартный ввод или input.txt

Вывод

стандартный вывод или output.txt

Напишите программу для генерации вертикальных столбчатых диаграмм в рамке.

Формат ввода

Вводятся несколько натуральных чисел на одной строке, разделённые пробелами.

Формат вывода

Выводятся вертикальные столбцы из знаков звёздочки *, высоты которых равны введённым числам, причём имеются дополнительные столбцы слева и справа от диаграммы и горизонтальная линия сверху, проведённая так, чтобы между ней и самым высоким из столбцов диаграммы был зазор ровно в один ряд.

Пример

Ввод	Вывод
3 5 4 2	***** * * * * * * * * * ***** ***** *****

```
1 n = list(map(int, input().split()))
2 xmax = len(n)
3 ymax = max(n)
4 print('*' * (xmax + 2))
5 print('*' + ' ' * xmax + '*')
6 for y in range(1, ymax + 1):
7     print(end='*')
8     for nk in n:
9         if nk >= ymax - y + 1:
10             print(end='*')
11         else:
12             print(end=' ')
13     print('*')
14
```

Пример 5. Джек-Победитель-Великанов

Ограничение времени

1 секунда

Ограничение памяти

64Mb

Ввод

стандартный ввод или input.txt

Вывод

стандартный вывод или output.txt

Имеется герой со множеством титулов. Нам нужно красиво записать все титулы героя.

Формат ввода

На первой строке вводится число n — количество титулов.

Далее следует n титулов. Каждый титул состоит из одного или нескольких слов, причём слова титула могут быть разбросаны на несколько строк, как в примере (но пустых строк нет). Конец титула отмечен звёздочкой на отдельной строке.

Формат вывода

Нужно вывести все титулы в одну, в порядке, обратном вводу, разделяя их запятыми, а слова в пределах одного титула — дефисами.

Пример

Ввод	Вывод
3 Просто Приятный Парень * Самый Продуктивный	Джек-Победитель-Великанов, Самый-Продуктивный- Выращиватель-Бобов, Просто-Приятный-Парень

Ввод	Вывод
Выращиватель Бобов * Джек Победитель Великанов *	

```

1  n = int(input())
2  s = []
3  c = ''
4  for i in range(n):
5      t = input()
6      while t != '*':
7          t = t.split()
8          t = '-'.join(t)
9          c = c + '-' + t
10         t = input()
11         c = c[1:len(c)]
12         c.split()
13         s.insert(0, c)
14         c = ''
15  print(' '.join(s))

```

Функции и методы работы со строками

Функция или метод	Назначение
S1 + S2	Конкатенация (сложение строк)
S1 * 3	Повторение строки
S[i]	Обращение по индексу
S[i:j:step]	Извлечение среза
len(S)	Длина строки
S.join(список)	Соединение строк из последовательности str через разделитель, заданный строкой
S1.count(S[, i, j])	количество вхождений подстроки s в строку s1. Результатом является число. Можно указать позицию начала поиска i и окончания поиска j
S.find(str, [start],[end])	Поиск подстроки в строке. Возвращает номер первого вхождения или -1
S.index(str, [start],[end])	Поиск подстроки в строке. Возвращает номер первого вхождения или вызывает ValueError
S.rindex(str, [start],[end])	Поиск подстроки в строке. Возвращает номер последнего вхождения или вызывает ValueError
S.replace(шаблон, замена)	Замена шаблона
S.split(символ)	Разбиение строки по разделителю
S.upper()	Преобразование строки к верхнему регистру
S.lower()	Преобразование строки к нижнему регистру

Задачи для практики (при решении используем функции и методы работы со строками):

1. Дана строка, состоящая ровно из двух слов, разделенных пробелом. Переставьте эти слова местами. Результат запишите в строку и выведите получившуюся строку. (Используйте метод строки *split*.)
2. Дана строка, содержащая русскоязычный текст. Найти количество слов, начинающихся с буквы "е".
3. В строке заменить все двоеточия (:) знаком процента (%). Подсчитать количество замен.
4. Определить, сколько раз в тексте встречается заданное слово.
5. Дана строка символов, среди которых есть одна открывающаяся и одна закрывающаяся скобки. Вывести на экран все символы, расположенные внутри этих скобок.